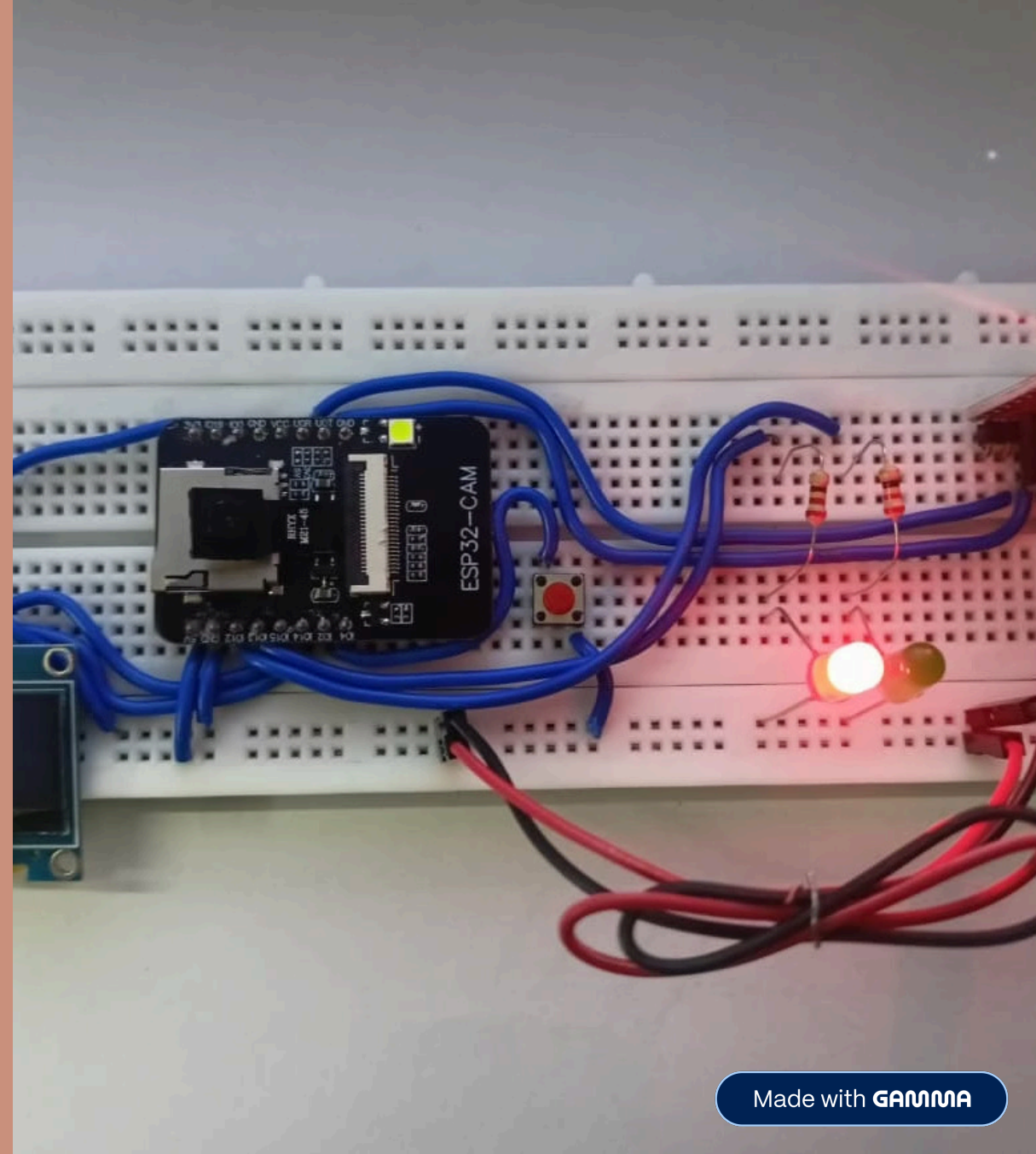


IoT-Based Smart Doorbell System

Using ESP32-CAM with Telegram Integration

Team Members:

- Pushti Bagla – 23T6149
- Suhani Lahoti – 23T6165
- Utkarsh Tayade – 23T6171
- Yash Bhoje – 23T6175



Project Overview

What It Does

This IoT-based system transforms a traditional doorbell into a smart security solution using ESP32-CAM. When a visitor presses the button, it captures an image and sends it via Telegram over WiFi.

Users can remotely grant or deny access through simple Telegram commands, while a hosted web server provides a live camera feed accessible from any browser.

Why It Matters

Key Features:

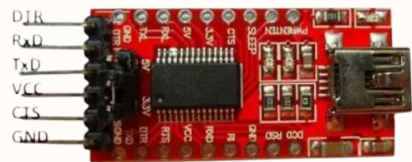
- Real-time visitor monitoring
- Remote access control
- Image capture and logging
- Live video streaming
- Cost-effective implementation

System Components



ESP32-CAM Module

Microcontroller with built-in WiFi and camera. Captures images, hosts web server, and serves as the core of the entire system.



FTDI Board

USB-to-Serial converter enabling programming and data transfer between ESP32-CAM and PC.



OLED Display

SSD1306 display showing system status: booting, WiFi connection, door open/closed states.



Push Button

Doorbell trigger that captures images and sends alerts when pressed.

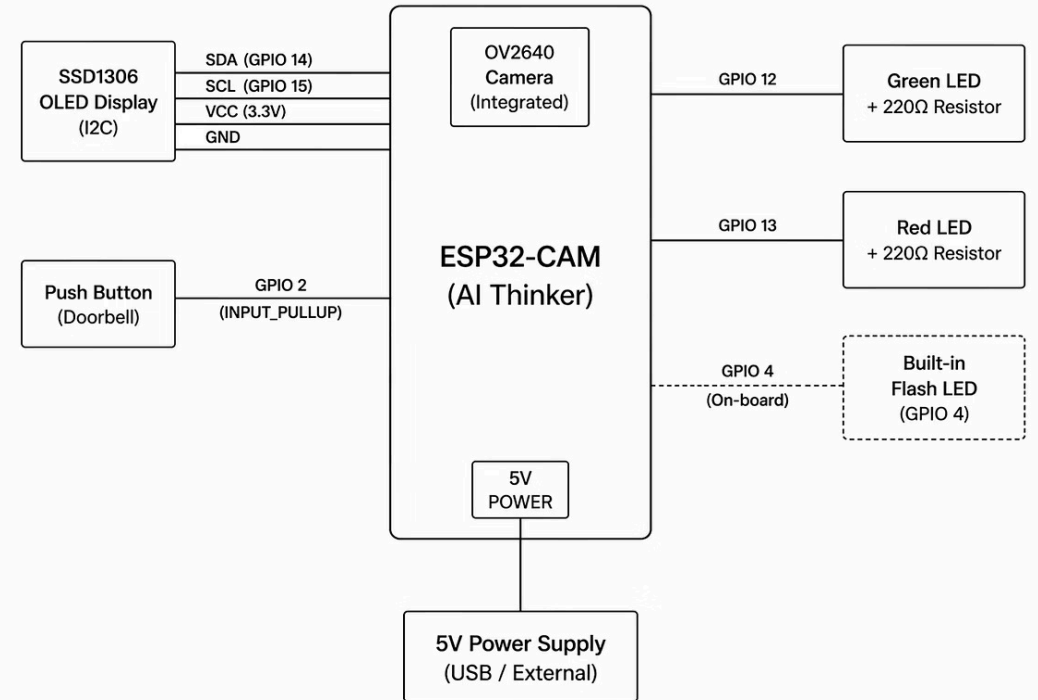


LED Indicators

Red and green LEDs indicating door status: open or closed/denied.

System Architecture

The schematic shows connections between ESP32-CAM, button, LEDs, OLED display, and power supply. The system uses GPIO pins for input/output control while maintaining WiFi connectivity for remote monitoring.



CODE

PART 1: Libraries, Definitions & Global Setup

```
#include "esp_camera.h"

#include <WiFi.h>

#include <WebServer.h>

#include "img_converters.h"

#include <WiFiClientSecure.h>

#include <UniversalTelegramBot.h>

// OLED LIBRARIES

#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

// ===== WIFI =====

const char* ssid = "BBH_4";

const char* password = "Bbhindore#1";

// ===== TELEGRAM =====

#define BOTtoken
"8722114401:AAESv1cHuPbc4syUrRm9Pm2HhDNsTL-imSc"

#define CHAT_ID "5205187464"

WiFiClientSecure client;

UniversalTelegramBot bot(BOTtoken, client);

// ===== LED =====

#define GREEN_LED 12

#define RED_LED 13

// ===== BUTTON =====

#define BUTTON_PIN 2

// ===== FLASH =====

#define FLASH_LED 4

// ===== OLED =====

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64 Adafruit_SSD1306

display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== CAMERA PINS =====

#define PWDN_GPIO_NUM 32

#define RESET_GPIO_NUM -1

#define XCLK_GPIO_NUM 0

#define SIOD_GPIO_NUM 26

#define SIOC_GPIO_NUM 27

#define Y9_GPIO_NUM 35

#define Y8_GPIO_NUM 34

#define Y7_GPIO_NUM 39

#define Y6_GPIO_NUM 36

#define Y5_GPIO_NUM 21

#define Y4_GPIO_NUM 19

#define Y3_GPIO_NUM 18

#define Y2_GPIO_NUM 5

#define VSYNC_GPIO_NUM 25

#define HREF_GPIO_NUM 23

#define PCLK_GPIO_NUM 22

WebServer server(80);
```

PART 2: Functions (OLED, Camera, Web, Telegram, Button, Serial)

```
// ===== OLED HELPER =====

void oledPrint(String msg) {

display.clearDisplay();

display.setCursor(0, 0);

display.println(msg);

display.display(); }

// ===== CAPTURE =====

void handle_capture() {

camera_fb_t * fb = esp_camera_fb_get();

if (!fb) {

server.send(500, "text/plain", "Camera capture failed");

return; }

uint8_t * bmp_buf = NULL;

size_t bmp_len = 0;

if (!frame2bmp(fb, &bmp_buf, &bmp_len)) { server.send(500,
"text/plain", "BMP conversion failed");
esp_camera_fb_return(fb);

return; }

server.setHeader("Content-Type", "image/bmp");
server.send_P(200, "image/bmp", (const char *)bmp_buf,
bmp_len);

free(bmp_buf); esp_camera_fb_return(fb); }

// ===== WEB PAGE ===== void
handle_root() {

String html = "";

html += "

ESP32-CAM Live

html += "";

html += "";

server.send(200, "text/html", html); }

// ===== SEND TO
TELEGRAM=====

void sendPhotoTelegram() {

// 📸 FLASH ON digitalWrite(FLASH_LED, HIGH);

delay(150);

String url = "http://" + WiFi.localIP().toString() + "/capture";
bot.sendMessage(CHAT_ID, "Visitor:\n" + url, "");

// 📸 FLASH OFF digitalWrite(FLASH_LED, LOW);

oledPrint("Doorbell Pressed"); }

// ===== TELEGRAM COMMANDS
=====

void handleTelegram() { int numNewMessages =
bot.getUpdates(bot.last_message_received + 1);

while (numNewMessages) {

for (int i = 0; i < numNewMessages; i++)

{ String text = bot.messages[i].text;
```

```
if (text == "/capture") {
sendPhotoTelegram();
}
if (text == "/open") {
digitalWrite(GREEN_LED, HIGH);
digitalWrite(RED_LED, LOW);
bot.sendMessage(CHAT_ID, "
Door Opened (10 sec)", "");

oledPrint("Door Opened");

delay(10000);

digitalWrite(GREEN_LED, LOW); digitalWrite(RED_LED, HIGH);
bot.sendMessage(CHAT_ID, " Door Closed", "");
oledPrint("Door Closed");

}

if (text == "/deny") {

digitalWrite(GREEN_LED, LOW); digitalWrite(RED_LED, HIGH);
bot.sendMessage(CHAT_ID, " Access Denied", "");
oledPrint("Access Denied"); }

}

numNewMessages =
bot.getUpdates(bot.last_message_received + 1);

}}

// ===== SERIAL TRIGGER =====

void handleSerialTrigger() {

if (Serial.available()) {

char input = Serial.read();

if (input == 'c') {
Serial.println("
Capture triggered"); sendPhotoTelegram(); }

}}

// ===== BUTTON FUNCTION
=====

void handleButtonPress()

{ static bool lastState = HIGH;

bool currentState = digitalRead(BUTTON_PIN);

if (lastState == HIGH && currentState == LOW)

{ Serial.println("🔔 Doorbell Pressed");

sendPhotoTelegram();
delay(300);

}

lastState = currentState; }
```

PART 3: Setup & Loop (Main Execution)

```
void setup() {

Serial.begin(115200);

pinMode(GREEN_LED, OUTPUT);

    pinMode(RED_LED, OUTPUT);

digitalWrite(GREEN_LED, LOW);

digitalWrite(RED_LED, HIGH);

// BUTTON INIT pinMode(BUTTON_PIN, INPUT_PULLUP);

// FLASH INIT pinMode(FLASH_LED, OUTPUT);
digitalWrite(FLASH_LED, LOW);

// OLED INIT Wire.begin(14, 15);

if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
Serial.println("OLED failed"); }

display.clearDisplay();

display.setTextSize(1);

display.setTextColor(WHITE);

oledPrint("Booting...");

// CAMERA CONFIG camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;

    config.ledc_timer = LEDC_TIMER_0;

config.pin_d0 = Y2_GPIO_NUM;

config.pin_d1 = Y3_GPIO_NUM;

config.pin_d2 = Y4_GPIO_NUM;

config.pin_d3 = Y5_GPIO_NUM;

    config.pin_d4 = Y6_GPIO_NUM;

    config.pin_d5 = Y7_GPIO_NUM;

config.pin_d6 = Y8_GPIO_NUM;

config.pin_d7 = Y9_GPIO_NUM;

config.pin_xclk = XCLK_GPIO_NUM;

config.pin_pclk = PCLK_GPIO_NUM;

config.pin_vsync = VSYNC_GPIO_NUM;

config.pin_href = HREF_GPIO_NUM;

config.pin_sccb_sda = SIOD_GPIO_NUM;
```

```
config.pin_sccb_scl = SIOC_GPIO_NUM;

config.pin_pwdn = PWDN_GPIO_NUM;

config.pin_reset = RESET_GPIO_NUM;

config.xclk_freq_hz = 20000000;

config.pixel_format = PIXFORMAT_RGB565;

config.frame_size = FRAMESIZE_QQVGA;

config.fb_count = 1;

if (esp_camera_init(&config) != ESP_OK) {

Serial.println("Camera init failed");

    oledPrint("Camera Failed");

return;

}

Serial.println("Camera initialized");

oledPrint("Camera Ready");

// WIFI WiFi.begin(ssid, password);

Serial.print("Connecting to WiFi");

oledPrint("Connecting WiFi");

while (WiFi.status() != WL_CONNECTED) { delay(500);
Serial.print("."); }

Serial.println("\n WiFi connected");
Serial.println(WiFi.localIP());

oledPrint("WiFi Connected");

// TELEGRAM client.setInsecure(); bot.sendMessage(CHAT_ID,
"🚀 System Ready", "");

oledPrint("System Ready");

// SERVER server.on("/", handle_root); server.on("/capture",
handle_capture); server.begin();

Serial.println("Server started"); Serial.println("Type 'c' to
capture image"); }

void loop() { server.handleClient(); handleTelegram();
handleSerialTrigger(); handleButtonPress(); }
```

Core Functionality



Visitor Detection

Button press triggers image capture with flash illumination



Image Capture

ESP32-CAM captures photo and converts to BMP format



Telegram Alert

Image URL sent instantly to user's Telegram chat



Access Control

User responds with /open or /deny command

Telegram Interface

Available Commands

/capture

Takes a photo and sends the visitor image with live URL

/open

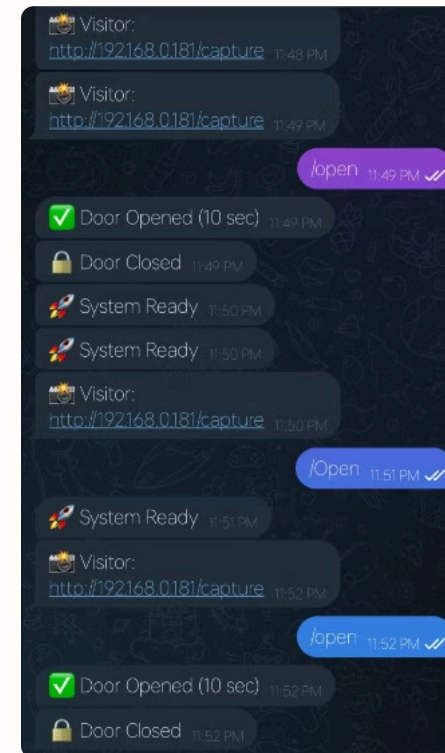
Activates green LED, opens door for 10 seconds, then closes

/deny

Keeps door closed, displays access denied message

Live Feed Access

Access live camera feed by visiting [http://\[IP_ADDRESS\]/](http://[IP_ADDRESS]/) in any web browser on the same network.



Conclusion & Project Summary

Project Summary

This IoT-based smart doorbell system successfully converts a traditional doorbell into a smart security solution using ESP32-CAM. The system enables real-time visitor monitoring, image capture, and remote access control through Telegram.

Real-time image capture and transmission

Web-based live video streaming

User-friendly Telegram interface

Cost-effective implementation

Scalable for homes and offices



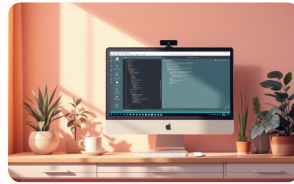
References

Key sources and tools used to build and document the smart doorbell system.



ESP32-CAM documentation

Official documentation and datasheet for hardware features and pinout details.



Arduino IDE

Programming environment used to write, compile, and upload the firmware.



Libraries and code

UniversalTelegramBot, Adafruit SSD1306, and the ESP32 camera web server example.



Communication protocols

WiFiClientSecure and HTTPS communication for secure remote connectivity.